



Introducción a gRPC

Cliente / Servidor

72.42 Programación de Objetos Distribuidos





Sistemas Distribuidos

Modelo de programación distribuido



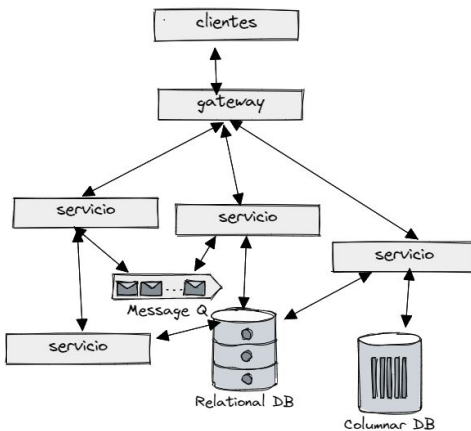
Sistema Distribuido - Necesidad

- Tanto el **modelo multiproceso** y el **modelo de programación concurrente** son dos modelos que pretenden paralelizar tareas dentro de un sistema.
- Estos modelos más allá de sus beneficios y problemas ya vistos tienen una cierta cantidad de limitantes dados por los recursos del sistema.
 - No se pueden correr concurrentemente más procesos de los que permite el procesador.
 - El número de Threads está limitado.
 - La memoria también.
 - Los procesos pueden llegar a competir por otros recursos compartidos como inodos, puertos, etc.

En algunos casos cuando se necesitan más recursos, potencia y/o eficiencia se puede utilizar el modelo distribuido

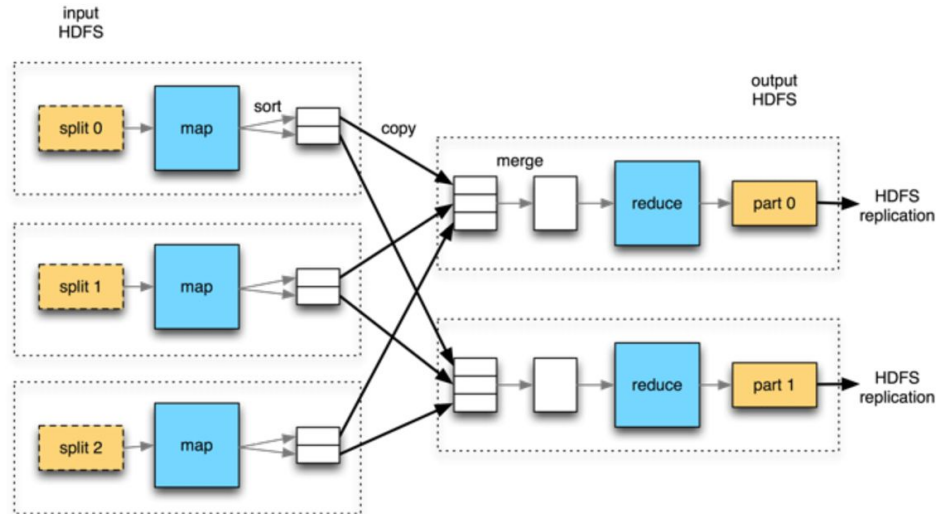
Sistema Distribuido

Es un **sistema computacional** cuyos componentes están **distribuidos en diferentes computadoras** (o elementos computacionales), que **se conectan y comunican** por medio de mensajes de una red. Este sistema se presenta ante sus usuarios como **un único sistema coherente**.



Sistema Distribuido - Objetivo

Las componentes del sistema **dividen una tarea** coordinando sus recursos **para completarla de manera más eficiente** que si se ejecutara en una sola computadora.





Cliente / Servidor

Modelo y Servicios





El modelo Cliente / Servidor

El modelo cliente servidor se puede considerar como **la unidad atómica en la comunicación** entre los nodos de un sistema distribuido.



El modelo Cliente / Servidor

A pesar de que pueden ser cientos o miles de máquinas **toda comunicación será entre dos máquinas** una actuando como **proveedor** de un servicio y la otra como **cliente** del mismo.

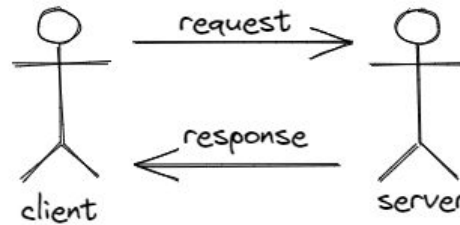
Servicio Ofrecido



Servicio requerido

¿Qué es un servicio?

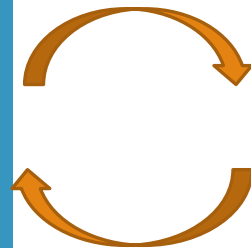
Parte de un sistema que **maneja recursos y presenta su funcionalidad a los usuarios** (personas o aplicaciones) restringiendo sus posibilidades a un **conjunto de operaciones** definidas en una **API**.



Servicios - Roles

Servidor

Proceso en una red de computadoras que **ejecuta las operaciones ofrecidas por la API** del servicio.



Cliente

Proceso en una red de computadoras que **solicita la ejecución de la operación** de un servicio.

El término **cliente** y el término **servidor** sólo **refiere al rol** que juega en un determinado momento de la existencia del servicio.



Servicios - Patrón de comunicación

La forma de implementar la comunicación entre cliente y servidor generalmente corresponden a dos estilos:

Request / Response

El estilo más común donde el cliente envía un pedido (*request*) y el servidor lo procesa **sincrónicamente** y una vez determinada la respuesta la envía hacia el cliente

Basado en Eventos

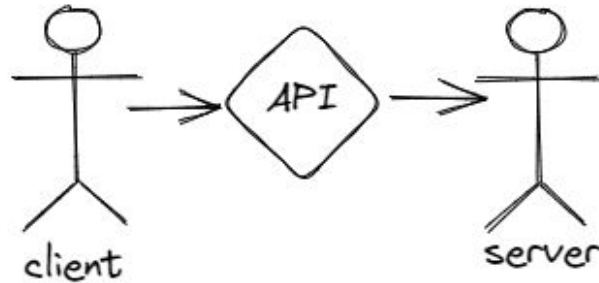
Es un tipo de comunicación **asincrónica** donde el cliente se registra para la recepción de cierto tipo de eventos y cuando estos ocurren el servidor se los envía.

La selección del estilo a usar dependerá mucho del tipo de servicio, cliente y tecnología.



Servicios - Interfaz

Una interfaz define un **conjunto de operaciones** que el servicio provee.





Servicios - Interfaz

La interfaz es el elemento más importante de un servicio ya que es el elemento que **le permite al cliente saber cómo invocar la operación** que requiere.

De cada operación su API/interfaz indica:

- modos de llamado
 - nombres
 - parámetros
 - valores de respuesta
 - errores.
- 



Servicios - Interfaz

Una **interfaz remota** puede definirse usando:

- **Herramientas específicas del lenguaje/framework:** por ejemplo interfaces remotas de Java para RMI.
 - **IDL** (lenguaje de definición de interfaces): por ejemplo **protobuf** **utilizado en gRPC** y otros.
 - **Documentación:** por ejemplo APIs Rest.
- 



IPC

Inter Process Communication





IPC - Comunicación entre procesos

¿Cómo programamos la comunicación entre 2 nodos?

Aplicaciones y Servicios

Sistemas Operativos

Computadoras y la Red

El acceso a estos recursos a nivel sistemas operativos y/o lenguajes de programación **es por medio de sockets.**



IPC - Mediante sockets

Aplicaciones y Servicios

Sistemas Operativos

Computadoras y la Red

- Para la comunicación a través de la red utilizamos TCP/IP y UDP/IP (punto a punto o *multicast* si es necesario). Entonces **el servicio está en** un nodo ubicado por su **dirección de IP** y el servicio en particular escucha en un **puerto** de dicho nodo.
- **Abriendo un socket** entre 2 nodos se envían los mensajes de *request* y *reply* de forma binaria.

Ejercicio 1: Sockets con Java

- Clonar el *template repo* <https://github.com/POD-ITBA/sockets>
- Abrirlo con un IDE
- Leer el **README.md**
- Ejecutar
 - `src/ar.edu.itba.pod.socket.server.GenericSocketServer` para levantar el servidor
 - El test `sendMessageTest` de `test/ar.edu.itba.pod.socket.client.GenericSocketClient` para que se establezca la comunicación

Servicios - IPC mediante sockets

El servidor tiene que abrir sockets y escuchar los mensajes.

```
public void start(int port) throws IOException {
    logger.info("starting server on port {}", port);
    try (ServerSocket server = new ServerSocket(port);
        Socket client = server.accept();
        var out = new PrintWriter(client.getOutputStream(), true);
        var in = new BufferedReader(new InputStreamReader(client.getInputStream())))
    {
        boolean loop = true;
        String inputLine;
        while (loop && (inputLine = in.readLine()) != null) {
            loop = handleClient(inputLine, out);
        }
    }
}
```

Servicios - IPC mediante sockets

En el servidor se contemplan dos operaciones: "1" para incrementar visitas y "." para finalizar la comunicación

```
private boolean handleClient(String inputLine, PrintWriter out) {  
    logger.debug("received message {}", inputLine);  
    if ("1".equals(inputLine)) {  
        visitCount++;  
    }  
    out.println(visitCount);  
    return !".".equals(inputLine);  
}
```

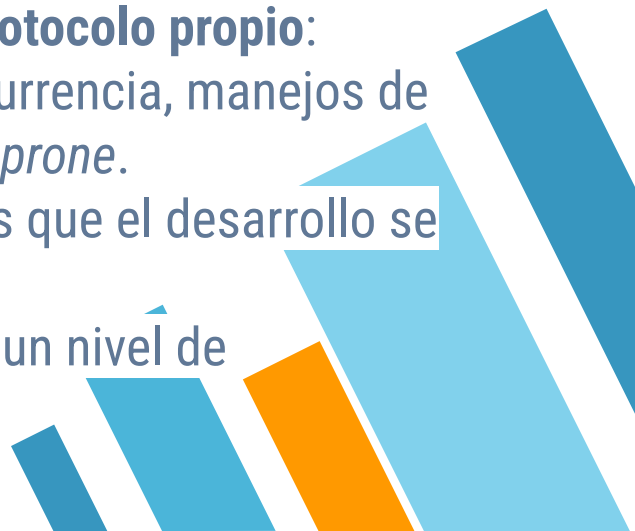
Servicios - IPC mediante sockets

El cliente se comunica y espera la respuesta.

```
public void startConnection(String ip, int port) throws IOException {  
    client = new Socket(ip, port);  
    out = new PrintWriter(client.getOutputStream(), true);  
    in = new BufferedReader(new InputStreamReader(client.getInputStream()));  
}  
  
public String sendMessage(String msg) throws IOException {  
    out.println(msg);  
    return in.readLine();  
}
```



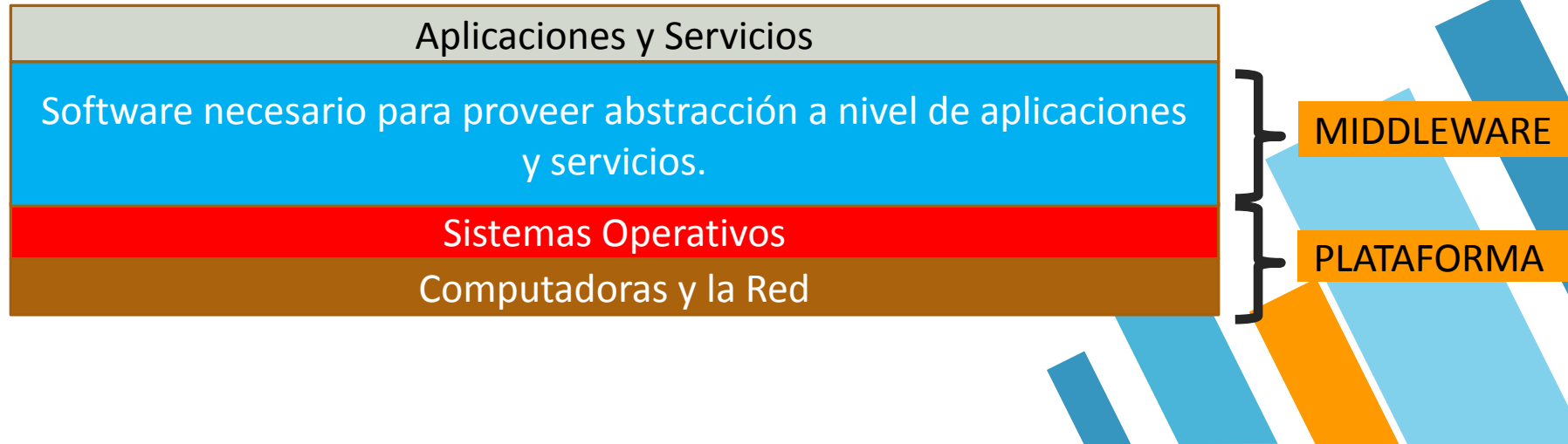
Dificultades IPC sockets

- La implementación no soporta **atender más de un cliente**
 - Si quisiéramos utilizar el servicio VisitCounter de clases anteriores ¿qué cambios habría que implementar? **¿Y si queremos soportar más operaciones?**
 - Con sockets es necesario definir y **mantener un protocolo propio**: serialización/parsing, múltiples conexiones, concurrencia, manejos de errores, etc. En resumen se vuelve confuso y *error prone*.
 - Una de las ideas principales al hacer un servicio es que el desarrollo se haga lo **más transparente posible**
 - **Para enfocarnos en la lógica de negocio** subimos un nivel de abstracción y **utilizaremos un *middleware***
- 

Servicios - Capas y Middleware

Se intenta separar la capa de aplicación de la de comunicación mediante **una capa intermedia llamada *middleware*** que se encarga de abstraer a la aplicación de todo lo requerido para realizar la comunicación por medio de una red.

Estos middlewares generalmente vienen provistos por medio de frameworks estandarizados.





RPC



RPC

Remote Procedure Call: es un protocolo que especifica el llamado de ejecución de funciones remotas sin tener que preocuparse por la comunicación entre el cliente y el servidor.



RPC

RPC define que habrá un **middleware** que se encarga de:

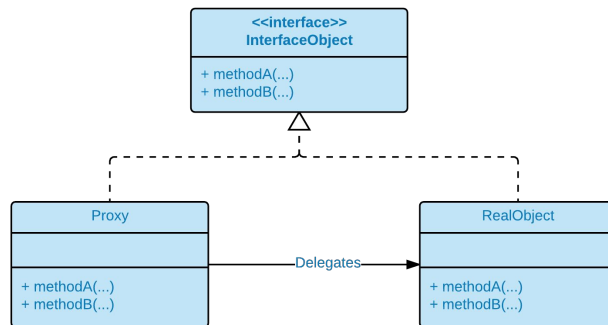
- **Establecer la comunicación** con el servidor, y **traducir el llamado al método y parámetros a una invocación por la red (Cliente)**
- Recibir la invocación, **llamar al método correspondiente y transformar la respuesta** a una comunicación por red para que la reciba el cliente (**Servidor**)
- Todo lo relativo a comunicación y errores



La intención de un framework como RPC es que **las llamadas al servicio sean lo más “transparentes” posibles**, es decir, que **no se diferencien de llamados locales**.

RPC - Middleware

Entonces estos elementos se encargan de la comunicación entre cliente y servidor aislando de los "problemas de comunicación" al cliente.



Efectivamente trabajando como un Proxy del servicio remoto.

Middleware: Definiciones

Stub:

Funciona de **proxy a un objeto remoto en el cliente**. Implementa los mismos métodos que la interfaz remota.

Su objetivo es recibir las invocaciones del cliente y pasarlas por la red (el método invocado y los parámetros) y esperar los resultados para leerlos y transmitirlos al cliente.

Middleware: Definiciones

Skeleton:

Se crea **en el servidor**, se **encarga de recibir el pedido del cliente**, interpretarlo, obtener los parámetros **y llamar al objeto remoto** con los mismos. Una vez que recibe la respuesta la transmite al cliente. Aislado de esta manera al objeto remoto de la forma de comunicación.

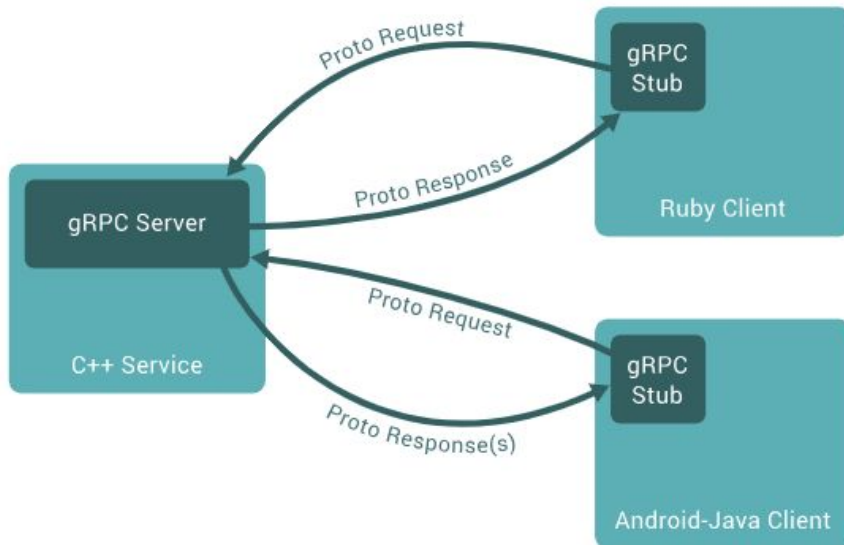
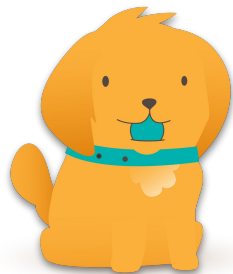


gRPC

gRPC

gRPC (2016) es un sistema de comunicación vía RPC *open source* desarrollado inicialmente por Google que apunta a ser:

- Multilenguaje
- Multiplataforma
- De alto rendimiento
- De uso general





gRPC

gRPC está basado en:

- **Protocol Buffer** para IDL y formato de serialización
- **HTTP/2** para el transporte.

y ofrece *built-in*:

- autenticación
 - streaming bidireccional
 - control de flujo
 - timeouts
 - cancelaciones
- 

Protocol Buffers

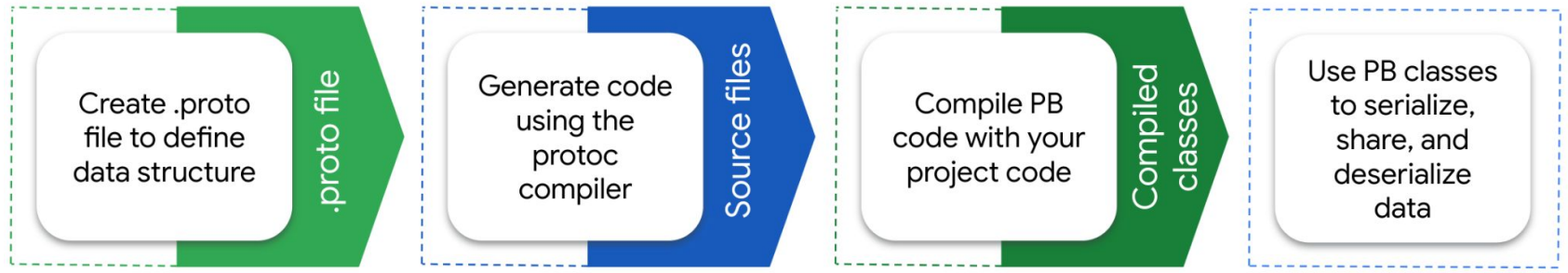
Protocol Buffer es un mecanismo para **serialización de datos** que es independiente de lenguaje y plataforma

- Al ser independiente del lenguaje es útil como IDL (lenguaje de definición de interfaces), es decir, definir servicios multilenguaje
- Además se puede utilizar como formato de serialización de la información que viaja entre el servicio y el cliente

Protocol Buffer

Como herramienta de **serialización** Protocol Buffer es

- **Tipado:** permite validar y optimizar por tipos
- **Binario:** Al no ser un protocolo textual es más rápido de leer y escribir para las máquinas (Machine readable).
- **Provee generadores:** para la serialización/deserialización en la mayoría de los lenguajes de programación.



Marshalización / Serialización

- **Serializar** un objeto significa convertir su estado en un *stream* de bytes que pueda luego ser utilizado para obtener una copia de la información del objeto.
- **Marshalizar:** codifica no sólo la información que contiene el objeto, sino que también la información de la definición del objeto (o cómo construirlo o al menos cómo encontrar la referencia a la definición).
- **De-serializar** un objeto significa tomar un *stream* de bytes (que corresponden a un objeto serializado previamente) y obtener a partir de él un objeto con la información que tenía el objeto original.
- **“Un-marshalling”** es similar a de-serializar, salvo que además permite obtener la definición del objeto.

En algunos lenguajes y situaciones los términos se utilizan como sinónimos



gRPC - Transporte

HTTP/2 es el protocolo de transporte. Es una mejora sobre el protocolo HTTP que permite generar aplicaciones más robustas, rápidas y eficientes.

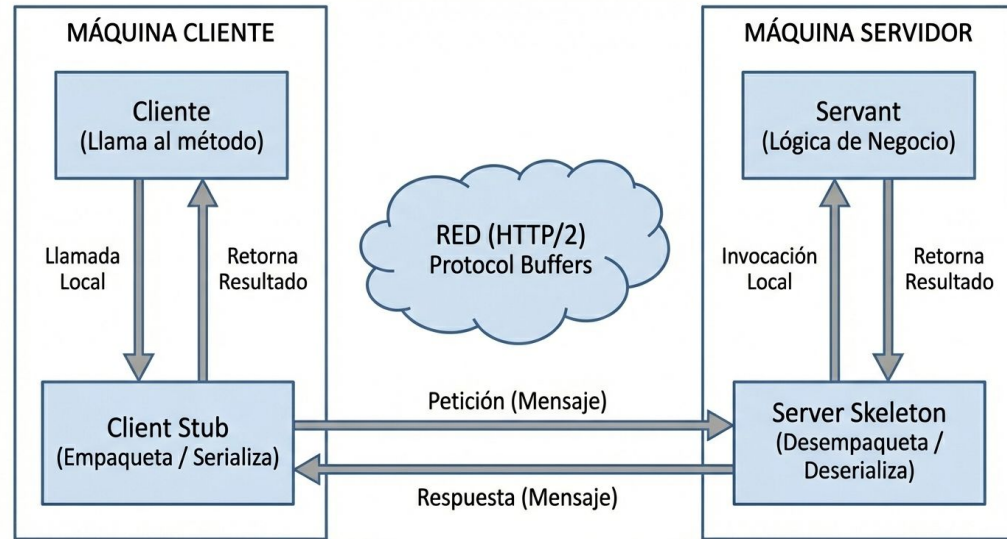
Sus principales características son:

- **Mantener la semántica de HTTP**
 - **Una única conexión “permanente”**
 - **Multiplexed streams**
 - **Server push**
 - **Compresión de HEADERS**
 - **Formato binario**
- 

gRPC - Flujo

Cuando un cliente llama a un método del servicio (utilizando el *stub*) el proceso que se realiza es:

1. El **stub** serializa los parámetros del método del servicio.
2. El **stub** genera un POST request usando HTTP/2, se genera la conexión y envía headers que indican el tipo de llamado, el método a correr remotamente, etc.
3. Se envía el mensaje por la red.
4. Al recibir el mensaje el servidor lee los headers para ver el método a llamar e invoca al **skeleton** correspondiente pasándole el mensaje.
5. El **skeleton** de-serializa los parámetros y realiza la llamada local a la implementación del **Servant** del método remoto.
6. Con la respuesta del **Servant** se genera el proceso inverso para que la respuesta le llegue al cliente





Spring gRPC

Spring gRPC (2025) es un *framework* que simplifica la implementación de servidores y clientes gRPC

Se basa en la implementación de gRPC Java

Ofrece:

- Inyección de dependencias para servicios y *stubs*
- *Starters* de Spring Boot para servidor y cliente
- Autoconfiguración vía propiedades
- *Testing* simplificado
- Integración con el ecosistema Spring



Ejercicio 2 - gRPC Greeter - Build

- Clonar el *template repo* <https://github.com/POD-ITBA/greeter>
- Abrirlo con un IDE
- Leer el **README.md**
- Analizar cómo está estructurado el proyecto
- Analizar el archivo **greeter.proto** en `api/src/main/proto`
- **Buildear** el proyecto siguiendo las instrucciones del README
- Analizar los archivos generados en **api/**

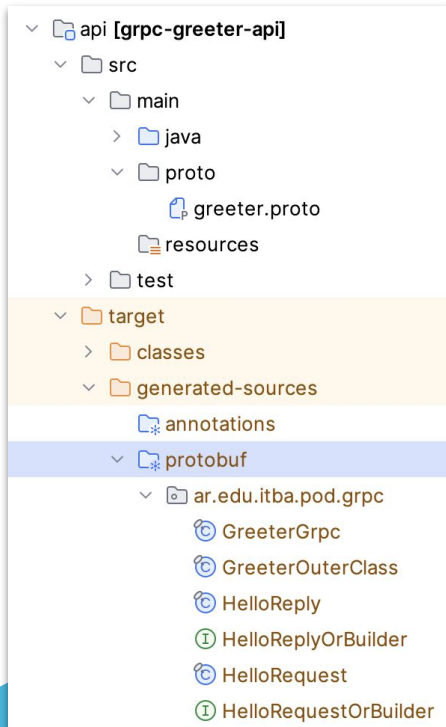
gRPC - Generación de código Java

En Java (como para todos los lenguajes) existe un compilador que recibe el archivo *.proto con la definición del servicio y se genera una clase llamada:

<NombreDeServicio>Grpc (GreeterGrpc)

La misma funciona como *middleware* y provee unas subclases que utilizan el servidor y el cliente para realizar la comunicación entre ellos.

Además se generarán clases por cada mensaje de entrada y salida.





Definición de Servicios

.proto





gRPC - Definición de Servicios

La definición de servicios se realiza en un archivo con extensión .proto
Antes del servicio se pueden customizar algunas características generales

```
syntax = "proto3";  
  
option java_multiple_files = true;  
option java_package = "ar.edu.itba.pod.grpc";
```



gRPC - Definición de Servicios

El servicio se define a partir de la palabra `service` y se listan sus métodos.

```
service Greeter {  
  // Sends a greeting  
  rpc SayHello (HelloRequest) returns (HelloReply);  
}
```

Todos los métodos **siempre** se prefijan con **rpc** y tienen exactamente **un mensaje de entrada** y **uno de salida**

gRPC - Definición de Servicios

Para terminar de definir el servicio se agregan los mensajes de entrada y salida

```
message HelloRequest {  
  string name = 1;  
}  
  
message HelloReply {  
  string message = 1;  
}
```

El mensaje debe existir aún si está vacío.

gRPC - Client

Para uso en el cliente se proveen 4 versiones de **stubs** para realizar llamados a los servicios del server:

- **<NombreDeServicio>BlockingStub**: Stub sincrónico
- **<NombreDeServicio>BlockingV2Stub**: *Checked exceptions*
- **<NombreDeServicio>FutureStub**: Stub asincrónico con *Future*
- **<NombreDeServicio>Stub**: Stub asincrónico con *observers*

Cada uno de ellos se instancia a partir de un *builder* y cuentan con los métodos del servicio variando sus parámetros y respuesta de acuerdo al tipo de llamado que realizan.

gRPC - Client

Para consumir el servicio desde el cliente se debe instanciar el *stub* deseado a partir de una `GrpcChannelFactory` y el nombre de un *channel*

```
@SpringBootApplication
public class Client {

    static void main(String[] args) {
        SpringApplication.run(Client.class, args);
    }

    @Bean
    GreeterGrpc.GreeterBlockingStub greeterStub(GrpcChannelFactory channels) {
        return GreeterGrpc.newBlockingStub(channels.createChannel("local"));
    }
}
```

gRPC - Client

Spring gRPC nos permite instanciar un `Client` sin indicar la configuración. En `client/src/main/resources/application.yml` se puede modificar el comportamiento por defecto (por ejemplo cambiar el puerto)

```
spring:
  grpc:
    client:
      channels:
        local:
          address: localhost:50051
        cloud:
          address: grpc-greeter-demo-server.XXX.norwayeast.azurecontainerapps.io:443
          negotiation-type: tls
  logging:
    level:
      root: info
```

gRPC - Lógica Cliente

Spring Boot nos permite ejecutar código registrando un @Bean de tipo CommandLineRunner. Se debe invocar el comando `spring-boot:run`

```
@SpringBootApplication
public class Client {
    ...

    @Bean
    CommandLineRunner run(GreeterGrpc.GreeterBlockingStub greeter) {
        return _ → {
            var name = "Foo";
            HelloRequest request = HelloRequest.newBuilder().setName(name).build();
            HelloReply response = greeter.sayHello(request);
            Log.info("Greeting: {}", response.getMessage());
        };
    }
}
```


Ejercicio 3 - gRPC Greeter - Cliente Cloud

- Implementar la clase **Client** que instancie un **BlockingStub** usando el *channel* “cloud”
- Modificar el `application.yml` para indicar la URL del servicio (Ver Link en Campus)
- Invocar al método **sayHello**
- Siguiendo las instrucciones del **README** en la sección Dev, **ejecutar:**
 - **Client**
- Verificar que obtiene el resultado esperado

gRPC - Server

Spring gRPC nos permite instanciar un Server sin indicar la configuración. En `server/src/main/resources/application.yml` se puede modificar el comportamiento por defecto (por ejemplo cambiar el puerto)

```
@SpringBootApplication
public class Server {

    static void main(String[] args) {
        SpringApplication.run(Server.class, args);
    }

}
```

```
spring:
  grpc:
    server:
      port: 50051
logging:
  level:
    root: info
```

gRPC - Lógica Servicio (Servant)

Para poder implementar el servicio se debe heredar de la clase **<NombreServicio>Grpc.<NombreServicio>ImplBase**.

Para que el Server lo reconozca, es necesaria la anotación `@Service`

```
@Service
public class Servant extends GreeterGrpc.GreeterImplBase {

}
```

Ejercicio 4 - gRPC Greeter - Cliente Local

- **Modificar** la clase **Client** para que el **BlockingStub** use el *channel* “local”
- **Modificar** la clase **Servant** para que extienda a `GreeterGrpc.GreeterImplBase`
- Siguiendo las instrucciones del **README** en la sección Dev, **ejecutar** en dos terminales distintas:
 - **Server**
 - **Client**
- ¿Qué resultado obtiene?

gRPC - Lógica Servicio (Servant)

Se deben **sobreescribir los métodos** deseados de la implementación base.
En este caso contaremos con un único método que será `void` y recibe dos parámetros:

- El message de *request*
- Un `StreamObserver` donde dejar las respuestas vía `onNext` y al terminar se utiliza el `onCompleted`. `onError` se invoca en caso de error.

```
public interface StreamObserver<V> {  
    void onNext(V value);  
  
    void onError(Throwable t);  
  
    void onCompleted();  
}
```

gRPC - Lógica Servicio (Servant)

Implementación completa del Servant:

```
@Service
public class Servant extends GreeterGrpc.GreeterImplBase {

    @Override
    public void sayHello(HelloRequest req, StreamObserver<HelloReply> responseObserver) {
        HelloReply reply = HelloReply.newBuilder()
            .setMessage("Hello " + req.getName())
            .build();
        responseObserver.onNext(reply);
        responseObserver.onCompleted();
    }
}
```

Ejercicio 5 - gRPC Greeter - Servant

- **Sobreescribir** el método `sayHello` de la clase **Servant**
- Siguiendo las instrucciones del **README** en la sección Dev, **ejecutar** en dos terminales distintas:
 - **Server**
 - **Client**
- Verificar que obtiene el resultado esperado

gRPC - Test Unitarios

En el proyecto de *sockets* teníamos el *test* deshabilitado porque se necesitaba contar con un servidor en ejecución. Spring gRPC puede crear un *server in-process*.

```
@SpringJUnit4Config(GreeterTest.TestConfig.class)
@AutoConfigureInProcessTransport
public class GreeterTest {
    @Autowired
    private GreeterGrpc.GreeterBlockingStub stub;
    @Test
    void sayHelloTest() {
        var name = "Foo";
        var request = HelloRequest.newBuilder().setName(name).build();
        assertEquals("Hello %s".formatted(name), stub.sayHello(request).getMessage());
    }
    @EnableAutoConfiguration
    @Import({ Servant.class })
    @ImportGrpcClients(types = GreeterGrpc.GreeterBlockingStub.class)
    static class TestConfig {
    }
}
```


gRPC - Client

@ImportGrpcClients evita que tengamos que crear el *bean* del *stub*.
Utiliza el *channel* indicado en `spring.grpc.client.default-channel`.

```
@SpringBootApplication
@ImportGrpcClients(types = GreeterGrpc.GreeterBlockingStub.class)
public class Client {

    static void main(String[] args) {
        SpringApplication.run(Client.class, args);
    }

    @Bean
    CommandLineRunner run(GreeterGrpc.GreeterBlockingStub greeter) {
        ...
    }
}
```

```
spring:
  grpc:
    client:
      channels:
        local:
          address: ...
      cloud:
          address: ...
          negotiation-type: tls
      default-channel:
        address: localhost:50051
  logging:
    level:
      root: info
```



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » protobuf.dev
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 